

MGQL: An Executable, Small-Step Semantics of GQL



Aditya Thimmaiah
The University of Texas at Austin



Tong-Nong Lin
The University of Texas at Austin



Milos Gligoric
The University of Texas at Austin

TEXAS
The University of Texas at Austin

Supported By amazon | science

NSF
CCF-2217696
CCF-2313027
CCF-2403036

Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

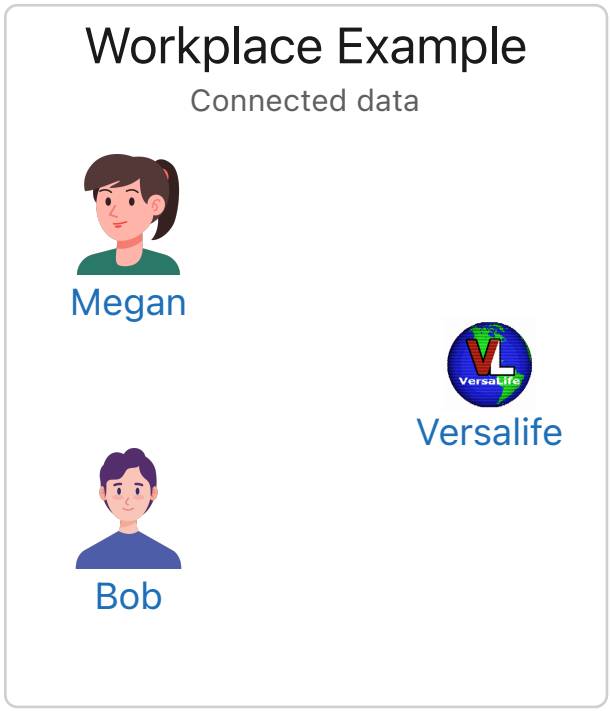
Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]



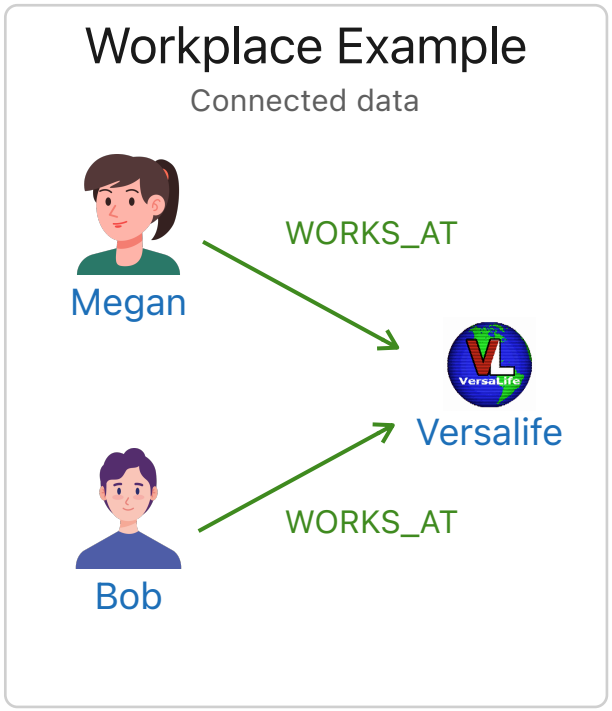
Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]



Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]



Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

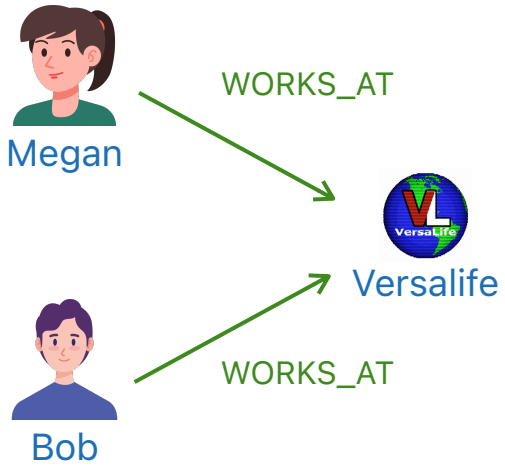
Relational Model

[Codd, CACM 1970]



Workplace Example

Connected data



Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

Relational Model

[Codd, CACM 1970]

Person

id	name
1	Megan
2	Bob

WORKS_AT

pid	cid
1	1
2	1

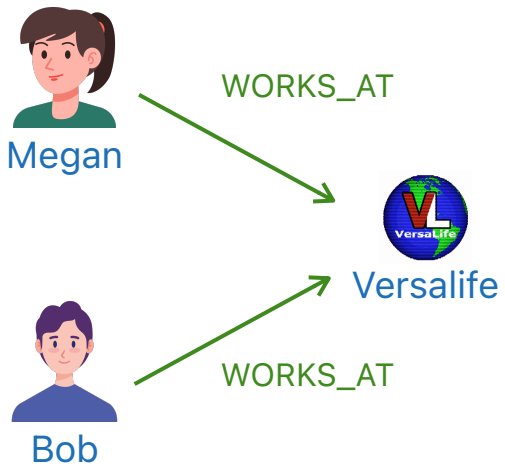
Company

id	name
1	Versalife

Only Relations

Workplace Example

Connected data



Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

Relational Model

[Codd, CACM 1970]

Person

id	name
1	Megan
2	Bob

WORKS_AT

pid	cid
1	1
2	1

Company

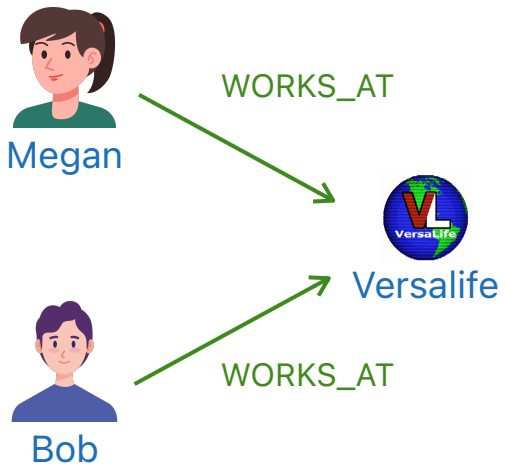
id	name
1	Versalife

× Relationships are not first-class

Only Relations

Workplace Example

Connected data



Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

Relational Model

[Codd, CACM 1970]

Person

id	name
1	Megan
2	Bob

WORKS_AT

pid	cid
1	1
2	1

Company

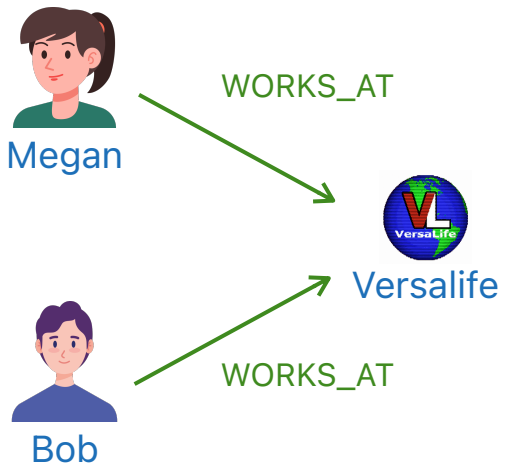
id	name
1	Versalife

× Relationships are not first-class

Only Relations

Workplace Example

Connected data



Property Graph Model

[Angles, AMW 2018]

Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

Relational Model

[Codd, CACM 1970]

Person

id	name
1	Megan
2	Bob

WORKS_AT

pid	cid
1	1
2	1

Company

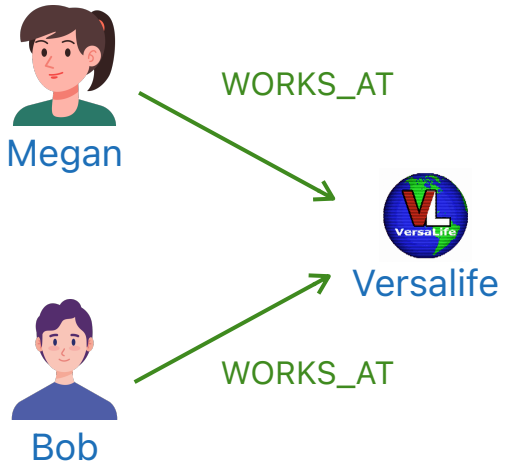
id	name
1	Versalife

× Relationships are not first-class

Only Relations

Workplace Example

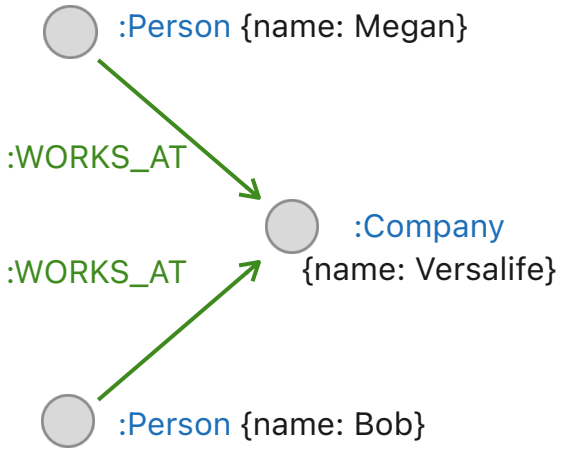
Connected data



Nodes & Edges

Property Graph Model

[Angles, AMW 2018]



Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

Relational Model

[Codd, CACM 1970]

Person

id	name
1	Megan
2	Bob

WORKS_AT

pid	cid
1	1
2	1

Company

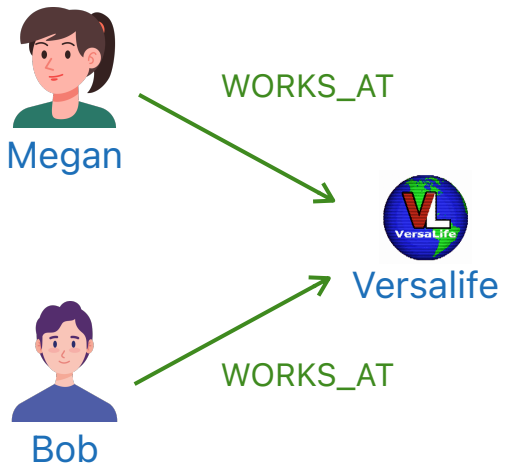
id	name
1	Versalife

× Relationships are not first-class

Only Relations

Workplace Example

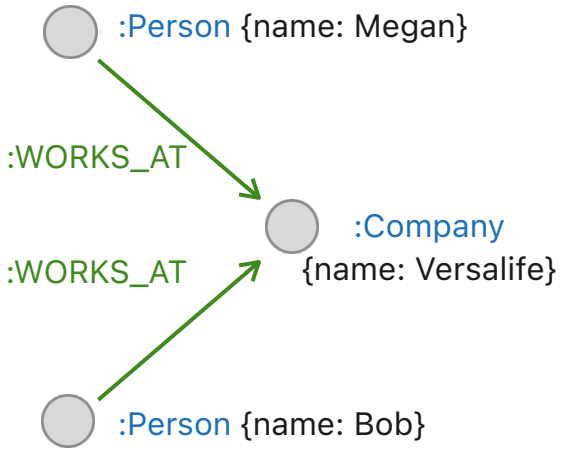
Connected data



Nodes & Edges

Property Graph Model

[Angles, AMW 2018]



Nodes and edges carry attributes

Background | Graph Databases

"A data model provides **conceptual tools** for representing real-world **entities** and the **relationships** among them."
[Angles & Gutierrez, CSUR 2008]

Relational Model

[Codd, CACM 1970]

Person

id	name
1	Megan
2	Bob

WORKS_AT

pid	cid
1	1
2	1

Company

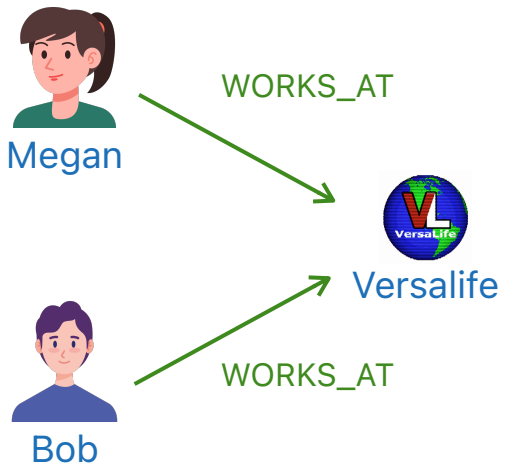
id	name
1	Versalife

× Relationships are not first-class

Only Relations

Workplace Example

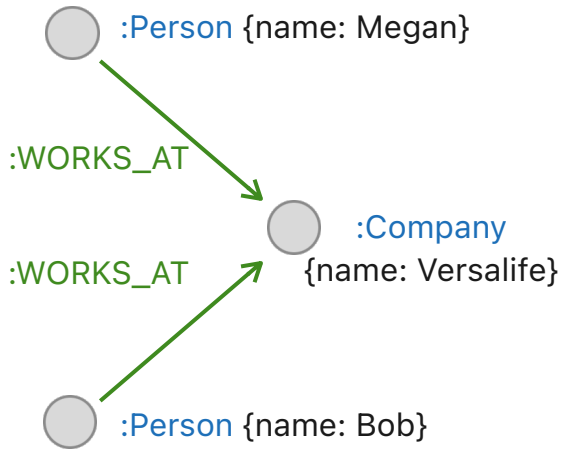
Connected data



Nodes & Edges

Property Graph Model

[Angles, AMW 2018]

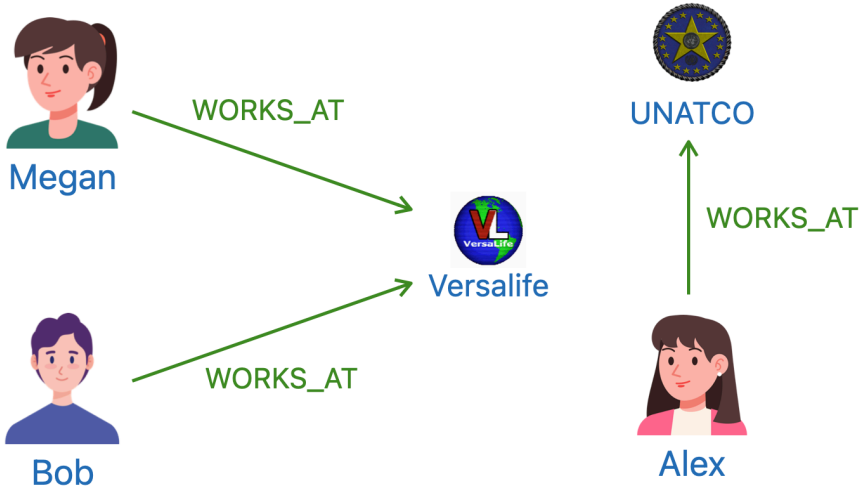


Nodes and edges carry attributes

Graph databases capture data better when relationships matter just as much as entities

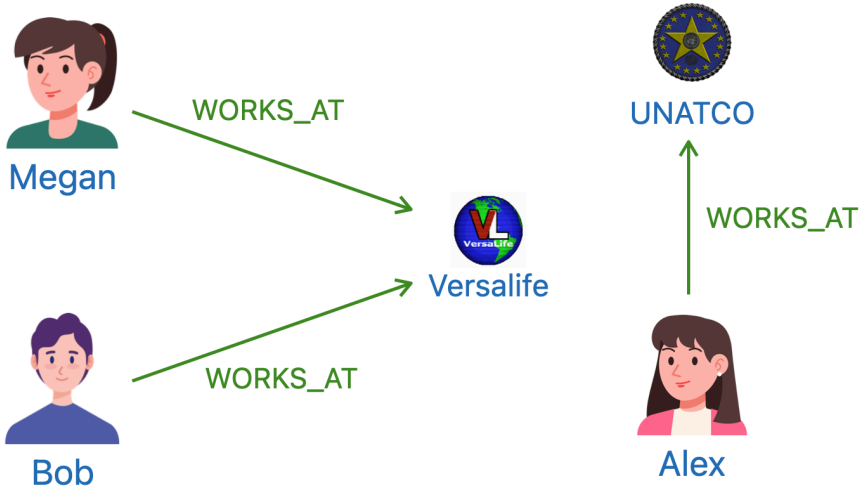
Motivation | Graph Queries

Graph



Motivation | Graph Queries

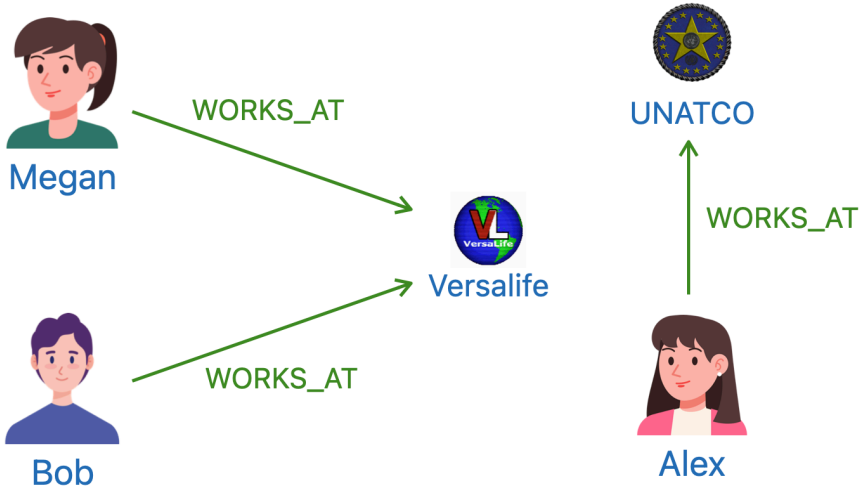
Graph



Who are **Megan's colleagues**?

Motivation | Graph Queries

Graph



Who are **Megan's colleagues**?

neo4j

```
MATCH (a:Person {name: 'Megan'})-[:WORKS_AT]->(c:Company)<-[:WORKS_AT]-(b:Person)
WHERE b <> a
RETURN b.name
```

Cypher

JanusGraph

```
g.V().has('Person', 'name', 'Megan').as('a').out('WORKS_AT').in('WORKS_AT')
 .where(neq('a')).values('name')
```

Gremlin

TigerGraph

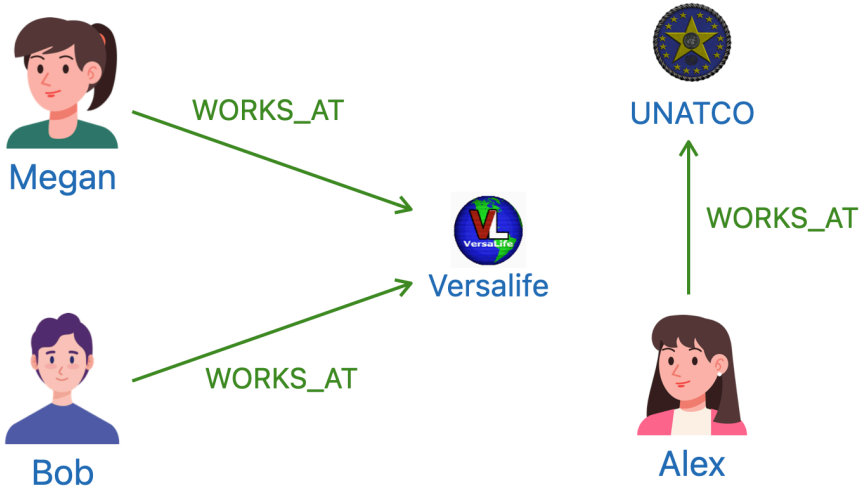
```
INTERPRET QUERY () SYNTAX v2 {
  R = SELECT b FROM Person:a -(WORKS_AT>)- Company:c-(<WORKS_AT)- Person:b
  WHERE a.name == "Megan" AND b != a; PRINT R;
}
```

GSQL

... and more: ULTIPA UQL, NebulaGraph nGQL

Motivation | Graph Queries

Graph



Who are **Megan's colleagues?**

neo4j

```
MATCH (a:Person {name: 'Megan'})-[:WORKS_AT]->(c:Company)<-[:WORKS_AT]-(b:Person)
WHERE b <> a
RETURN b.name
```

Cypher

JanusGraph

```
g.V().has('Person', 'name', 'Megan').as('a').out('WORKS_AT').in('WORKS_AT')
 .where(neq('a')).values('name')
```

Gremlin

TigerGraph

```
INTERPRET QUERY () SYNTAX v2 {
  R = SELECT b FROM Person:a -(WORKS_AT>)- Company:c-(<WORKS_AT)- Person:b
  WHERE a.name == "Megan" AND b != a; PRINT R;
}
```

GSQL

... and more: ULTIPA UQL, NebulaGraph nGQL

Same question, many languages

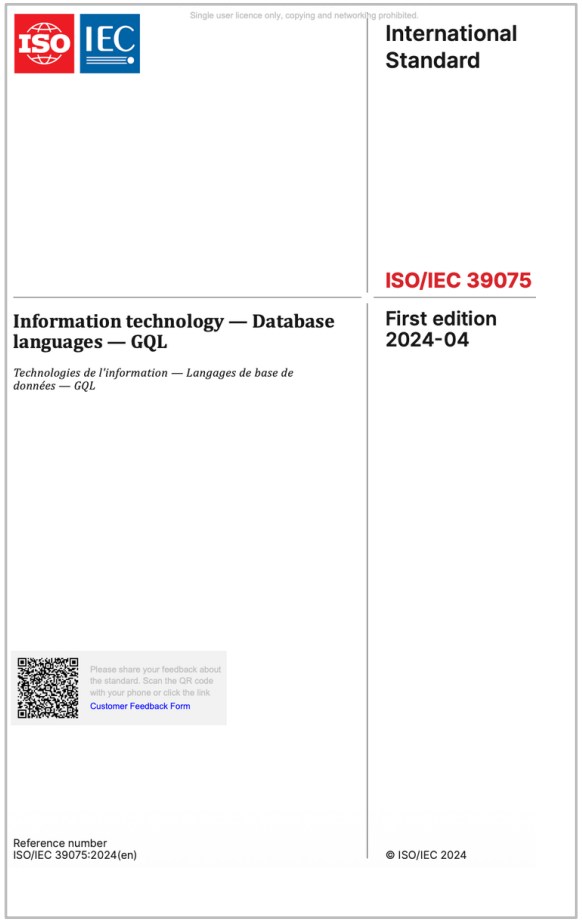
Motivation | Informal Standard

There was no standard for graph query languages…

Motivation | Informal Standard

There was no standard for graph query languages…

Until **2024**: ISO/IEC 39075, the Graph Query Language (**GQL**)



Motivation | Informal Standard

There was no standard for graph query languages…

Until **2024**: ISO/IEC 39075, the Graph Query Language (**GQL**)

neo4j — **Cypher**

> *"Cypher supports **the majority of mandatory GQL features**."*

Neo4j Cypher Manual 5 – GQL conformance

TigerGraph — **GSQL**

> *"TigerGraph currently **supports the path pattern matching syntax** (the MATCH or FROM clause) **of GQL**."*

GSQL Reference 4.3 – Querying Choices

NebulaGraph — **nGQL**

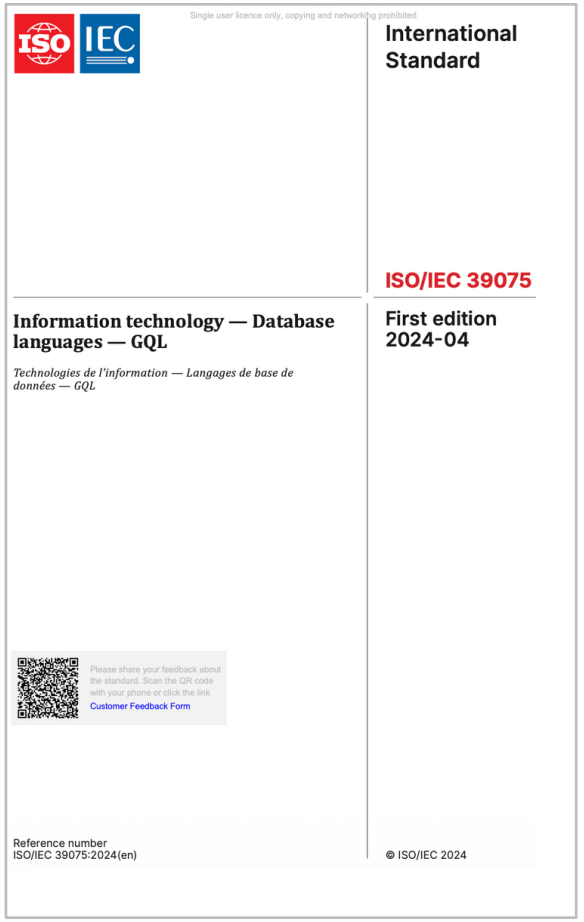
> *"NebulaGraph Enterprise v5.0 is the first and now the only distributed graph database that **implements the ISO/IEC 39075: GQL**…"*

NebulaGraph blog – v5.0 preview, Jun 2024

ULTIPA — **UQL**

> *"Ultipa now supports **nearly all GQL features**."*

Ultipa Docs – GQL Conformance



Motivation | Informal Standard

There was no standard for graph query languages…

Until **2024**: ISO/IEC 39075, the Graph Query Language (**GQL**)

neo4j — **Cypher**

> *"Cypher supports **the majority of mandatory GQL features**."*

Neo4j Cypher Manual 5 – GQL conformance

TigerGraph — **GSQL**

> *"TigerGraph currently **supports the path pattern matching syntax** (the MATCH or FROM clause) **of GQL**."*

GSQL Reference 4.3 – Querying Choices

NebulaGraph — **nGQL**

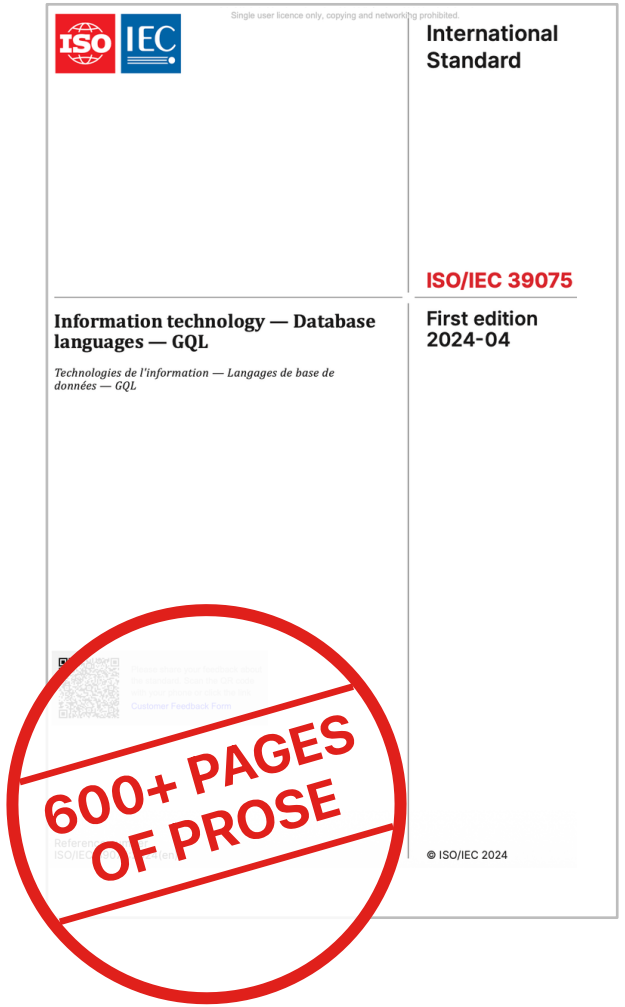
> *"NebulaGraph Enterprise v5.0 is the first and now the only distributed graph database that **implements the ISO/IEC 39075: GQL**…"*

NebulaGraph blog – v5.0 preview, Jun 2024

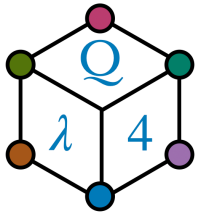
ULTIPA — **UQL**

> *"Ultipa now supports **nearly all GQL features**."*

Ultipa Docs – GQL Conformance



Contribution | Enable Formal Reasoning



MGQL

An executable, small-step semantics of GQL, mechanized in Lean 4

The first mechanized semantics for a substantial read-only fragment of GQL

enables

Checking correctness

Is this engine's answer the one the standard prescribes?

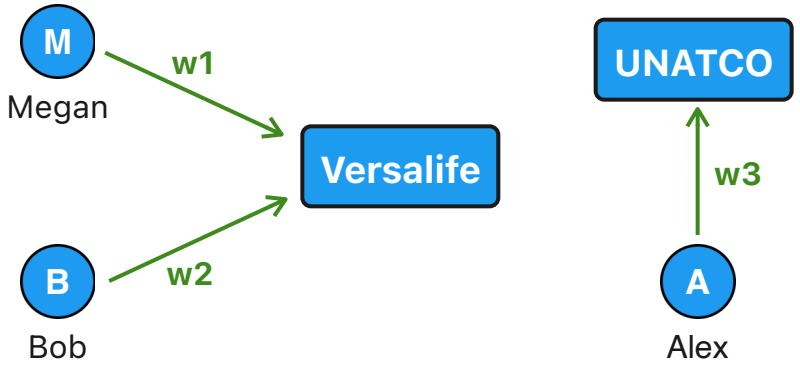
Query equivalence

Do these two queries return the same result on every graph?

GQL Example

"RETURN b.name" is not valid GQL, it should be aliased with "AS"
I am going to ignore this in these examples to keep things simple

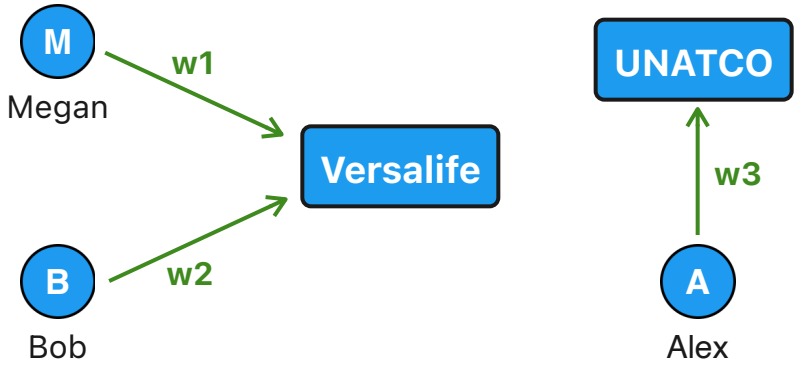
Graph G



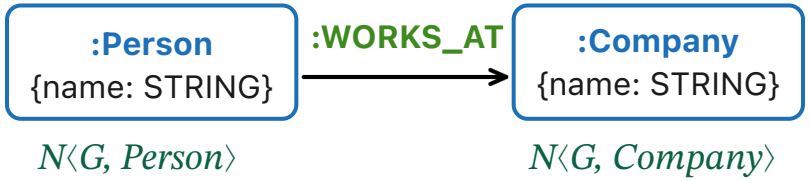
GQL Example

"RETURN b.name" is not valid GQL, it should be aliased with "AS"
I am going to ignore this in these examples to keep things simple

Graph G



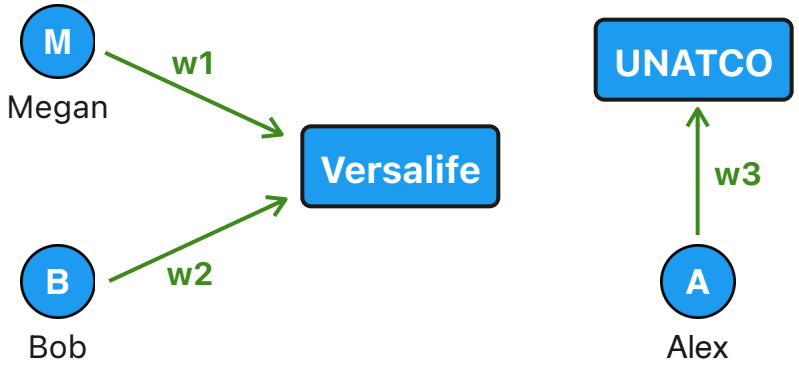
Schema of G



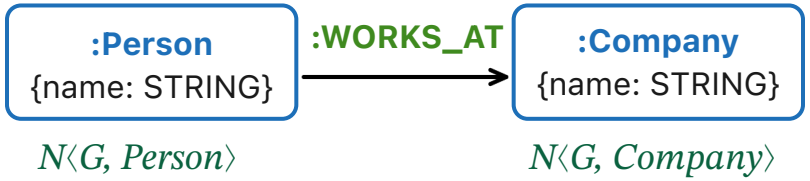
GQL Example

"RETURN b.name" is not valid GQL, it should be aliased with "AS"
I am going to ignore this in these examples to keep things simple

Graph G



Schema of G

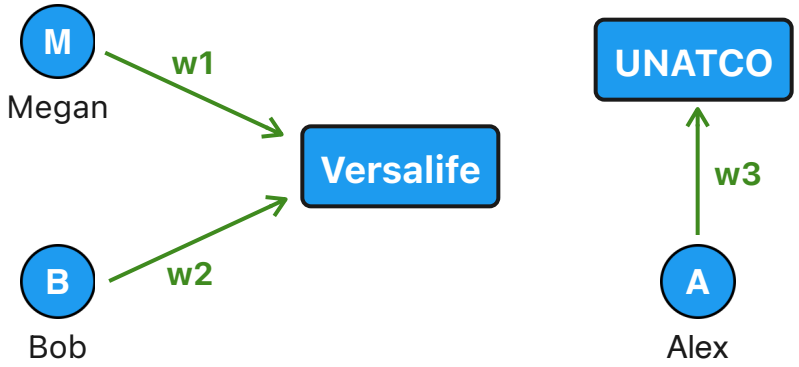



```
USE G MATCH (a {name: 'Megan'})-[:WORKS_AT]->(c)<-[:WORKS_AT]-(b) RETURN b.name
```

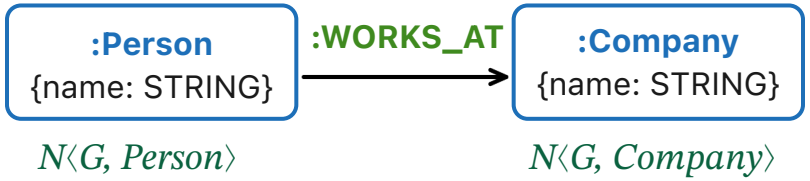
GQL Example

"RETURN b.name" is not valid GQL, it should be aliased with "AS"
I am going to ignore this in these examples to keep things simple

Graph G



Schema of G




```
USE G MATCH (a {name: 'Megan'})-[:WORKS_AT]->(c)<-[:WORKS_AT]-(b) RETURN b.name
```

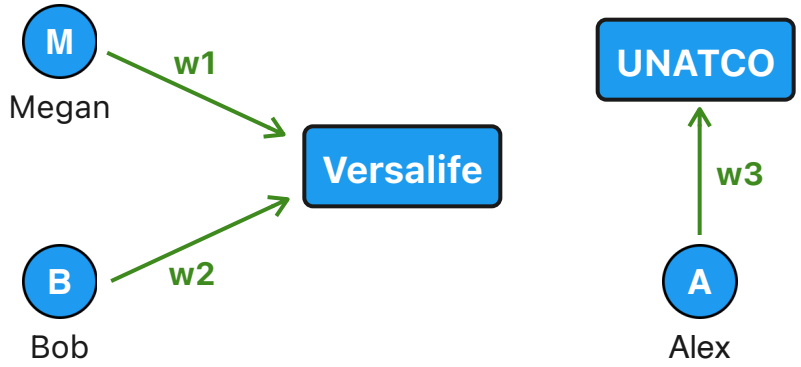
USE G

pick the graph

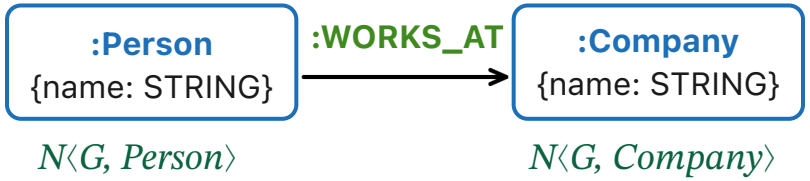
GQL Example

"RETURN b.name" is not valid GQL, it should be aliased with "AS"
I am going to ignore this in these examples to keep things simple

Graph G



Schema of G

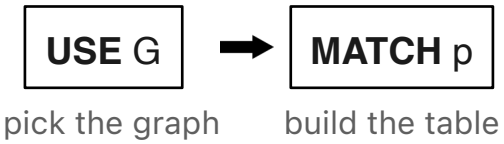



```
USE G MATCH (a {name: 'Megan'})-[:WORKS_AT]->(c)<-[:WORKS_AT]-(b) RETURN b.name
```

Matched Binding Table (B)

variables	a	c	b
schema (Γ)	$N\langle G, Person\rangle$	$N\langle G, Company\rangle$	$N\langle G, Person\rangle$
records (R)	M	Versalife	B
	M	Versalife	M

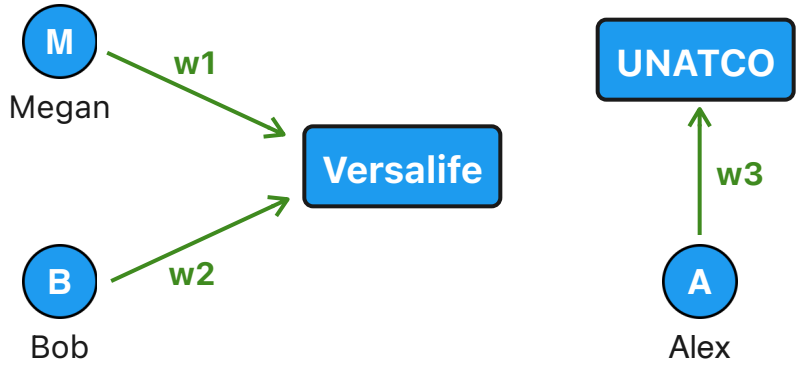
Row 2 comes back to Megan over the **same** edge w1.
Is she her own colleague?



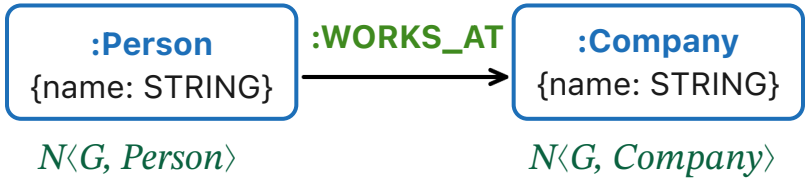
GQL Example

"RETURN b.name" is not valid GQL, it should be aliased with "AS"
I am going to ignore this in these examples to keep things simple

Graph G



Schema of G




```
USE G MATCH (a {name: 'Megan'})-[:WORKS_AT]->(c)<-[:WORKS_AT]-(b) RETURN b.name
```

Matched Binding Table (B)

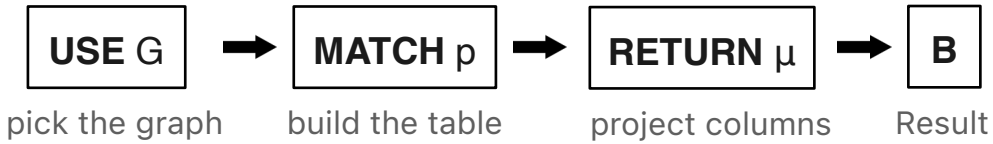
variables	a	c	b
schema (Γ)	$N\langle G, Person\rangle$	$N\langle G, Company\rangle$	$N\langle G, Person\rangle$
records (R)	M	Versalife	B
	M	Versalife	M

Row 2 comes back to Megan over the **same** edge w1.
Is she her own colleague?

result Binding Table (B)

b.name
STRING
Bob
Megan?

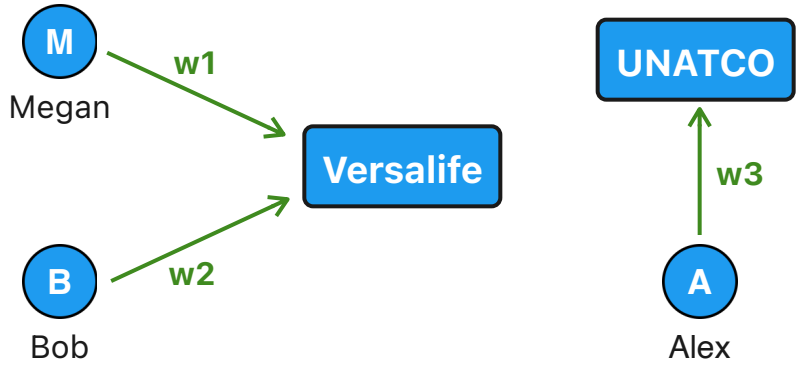
RETURN b.name
projects column b



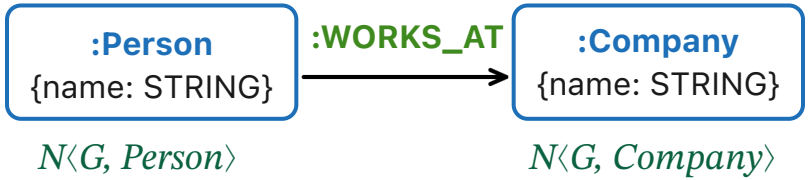
GQL Example

"RETURN b.name" is not valid GQL, it should be aliased with "AS"
I am going to ignore this in these examples to keep things simple

Graph G



Schema of G




```
USE G MATCH (a {name: 'Megan'})-[:WORKS_AT]->(c)<-[:WORKS_AT]-(b) RETURN b.name
```

Matched Binding Table (B)

variables	a	c	b
schema (Γ)	$N\langle G, Person\rangle$	$N\langle G, Company\rangle$	$N\langle G, Person\rangle$
records (R)	M	Versalife	B
	M	Versalife	M

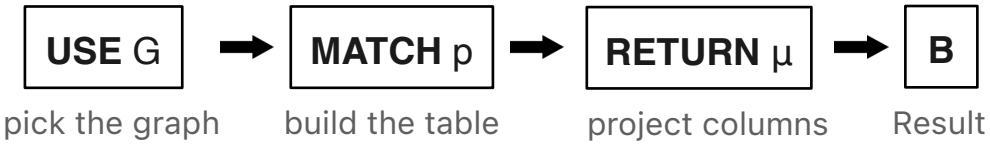
Row 2 comes back to Megan over the **same** edge w1.
Is she her own colleague?

result Binding Table (B)

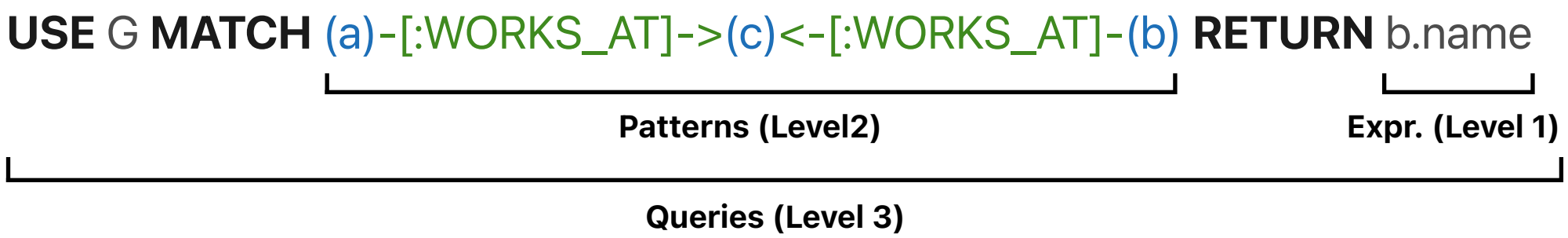
b.name
STRING
Bob
Megan?

RETURN b.name
projects column b

Every clause transforms a binding table



Main Insight | Hierarchical Design



Level 1 – Expressions

Constants, property access, predicates, aggregates

Level 2 – Patterns

Three sub-levels: **atoms** (no context) → **path patterns** (internal context) → **pattern lists** (surrounding context)

Level 3 – Queries

USE / MATCH / WHERE / RETURN, then composite queries (UNION, ...)

Each level is defined only in terms of the level below, so rules and proofs compose

Main Insight | Typing & Semantics Mirror

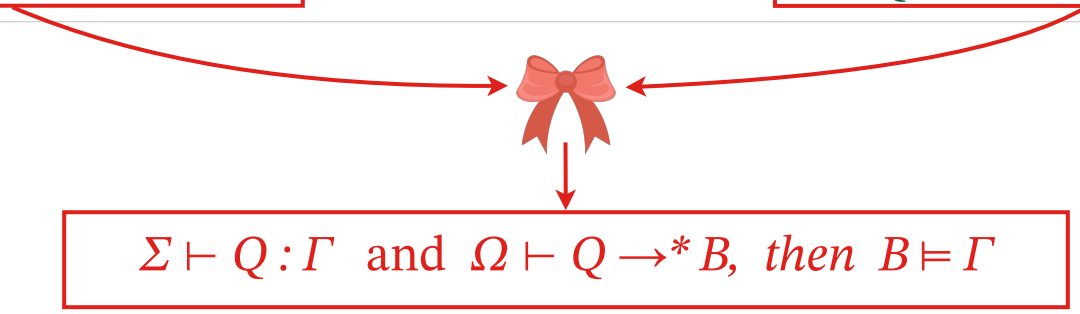
Constructs	Static (typing)		Dynamic (small-step semantics)
Atoms	filter the **schemas** by a label expression $\varsigma \vdash l \twoheadrightarrow \varsigma'$	$\longleftrightarrow$	check an **element's** labels against it $\iota \vdash l \Downarrow b$
Path steps	path concatenation: **schema** compatibility $\theta(\zeta_1, \xi_2, \zeta_3)$	$\longleftrightarrow$	path concatenation: **endpoints** must meet $P_1 \cdot \eta(B) \cdot P_2$
Quantifiers	a variable's type becomes a **list type** $\Gamma\uparrow^K(x) = List\ \Gamma(x)$	$\longleftrightarrow$	bindings accumulate into a **list** $(R_1 \bigstar^K R_2)(x) = R_1(x) :: [R_2(x)]$
Queries	query Q gets a result **table schema** $\Sigma \vdash Q : \Gamma$	$\longleftrightarrow$	query Q reduces to a **binding table** $\Omega \vdash Q \longrightarrow^* B$

Main Insight | Typing & Semantics Mirror

Constructs	Static (typing)		Dynamic (small-step semantics)
Atoms	filter the **schemas** by a label expression $\varsigma \vdash l \twoheadrightarrow \varsigma'$	$\longleftrightarrow$	check an **element's** labels against it $\iota \vdash l \Downarrow b$
Path steps	path concatenation: **schema** compatibility $\theta(\zeta_1, \xi_2, \zeta_3)$	$\longleftrightarrow$	path concatenation: **endpoints** must meet $P_1 \cdot \eta(B) \cdot P_2$
Quantifiers	a variable's type becomes a **list type** $\Gamma\uparrow^K(x) = List\ \Gamma(x)$	$\longleftrightarrow$	bindings accumulate into a **list** $(R_1 \bigstar^K R_2)(x) = R_1(x) :: [R_2(x)]$
Queries	query Q gets a result **table schema** $\Sigma \vdash Q : \Gamma$	$\longleftrightarrow$	query Q reduces to a **binding table** $\Omega \vdash Q \longrightarrow^* B$

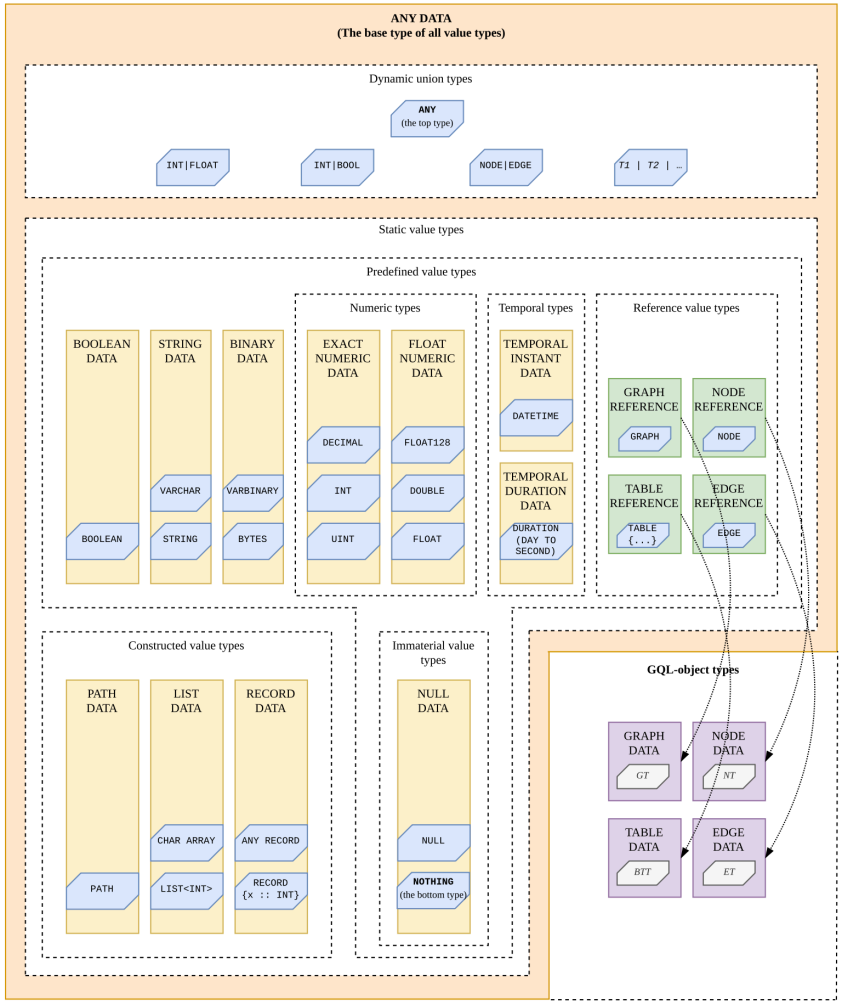
Main Insight | Typing & Semantics Mirror

Constructs	Static (typing)		Dynamic (small-step semantics)
Atoms	filter the **schemas** by a label expression $\varsigma \vdash l \twoheadrightarrow \varsigma'$	$\longleftrightarrow$	check an **element's** labels against it $\iota \vdash l \Downarrow b$
Path steps	path concatenation: **schema** compatibility $\theta(\zeta_1, \xi_2, \zeta_3)$	$\longleftrightarrow$	path concatenation: **endpoints** must meet $P_1 \cdot \eta(B) \cdot P_2$
Quantifiers	a variable's type becomes a **list type** $\Gamma\uparrow^K(x) = List\ \Gamma(x)$	$\longleftrightarrow$	bindings accumulate into a **list** $(R_1 \bigstar^K R_2)(x) = R_1(x) :: [R_2(x)]$
Queries	query Q gets a result **table schema** $\Sigma \vdash Q : \Gamma$	$\longleftrightarrow$	query Q reduces to a **binding table** $\Omega \vdash Q \rightarrow^* B$



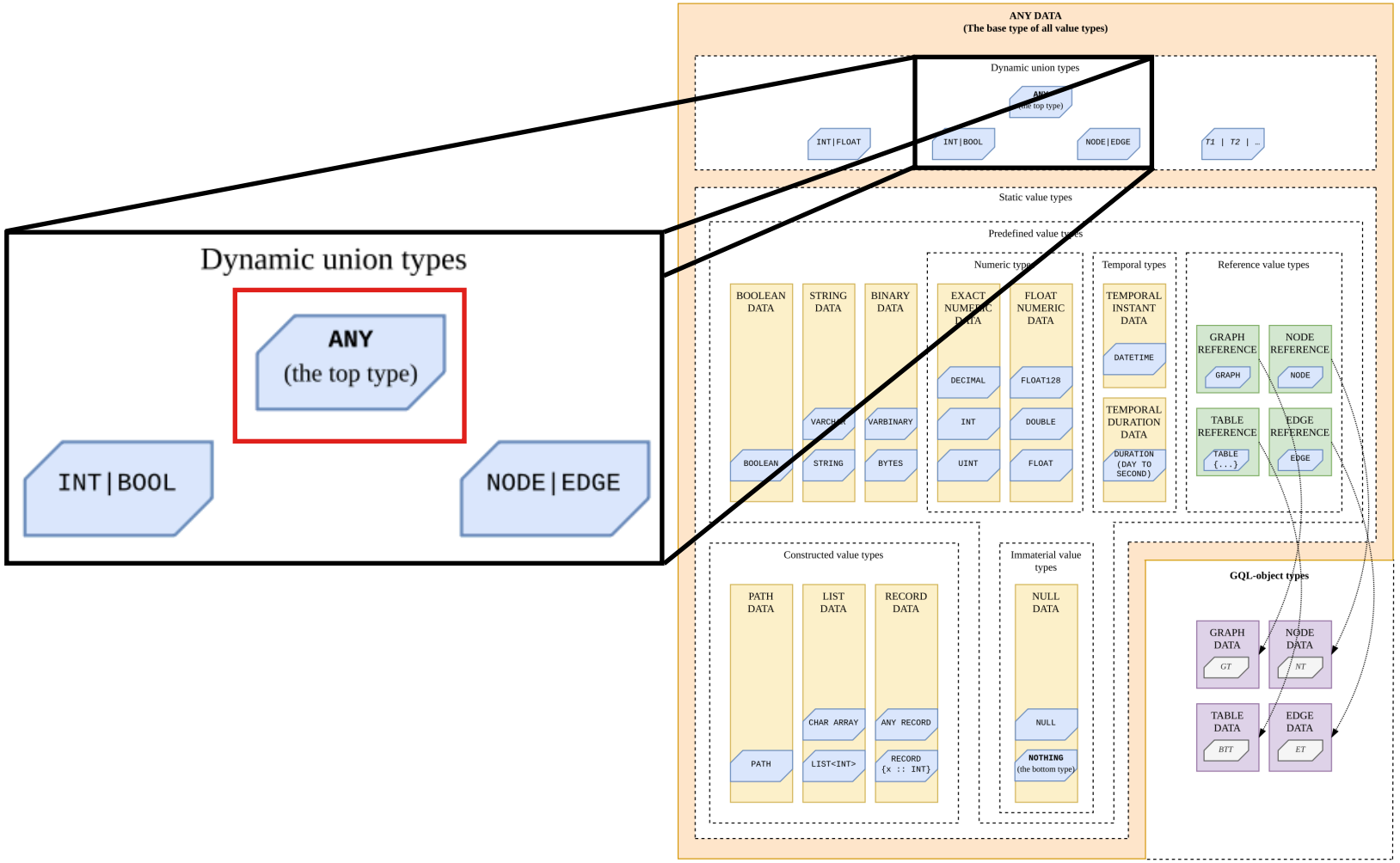
$$\Sigma \vdash Q : \Gamma \ \text{ and } \ \Omega \vdash Q \rightarrow^* B, \ \textit{then} \ \ B \vDash \Gamma$$

Type System | The Empty Former



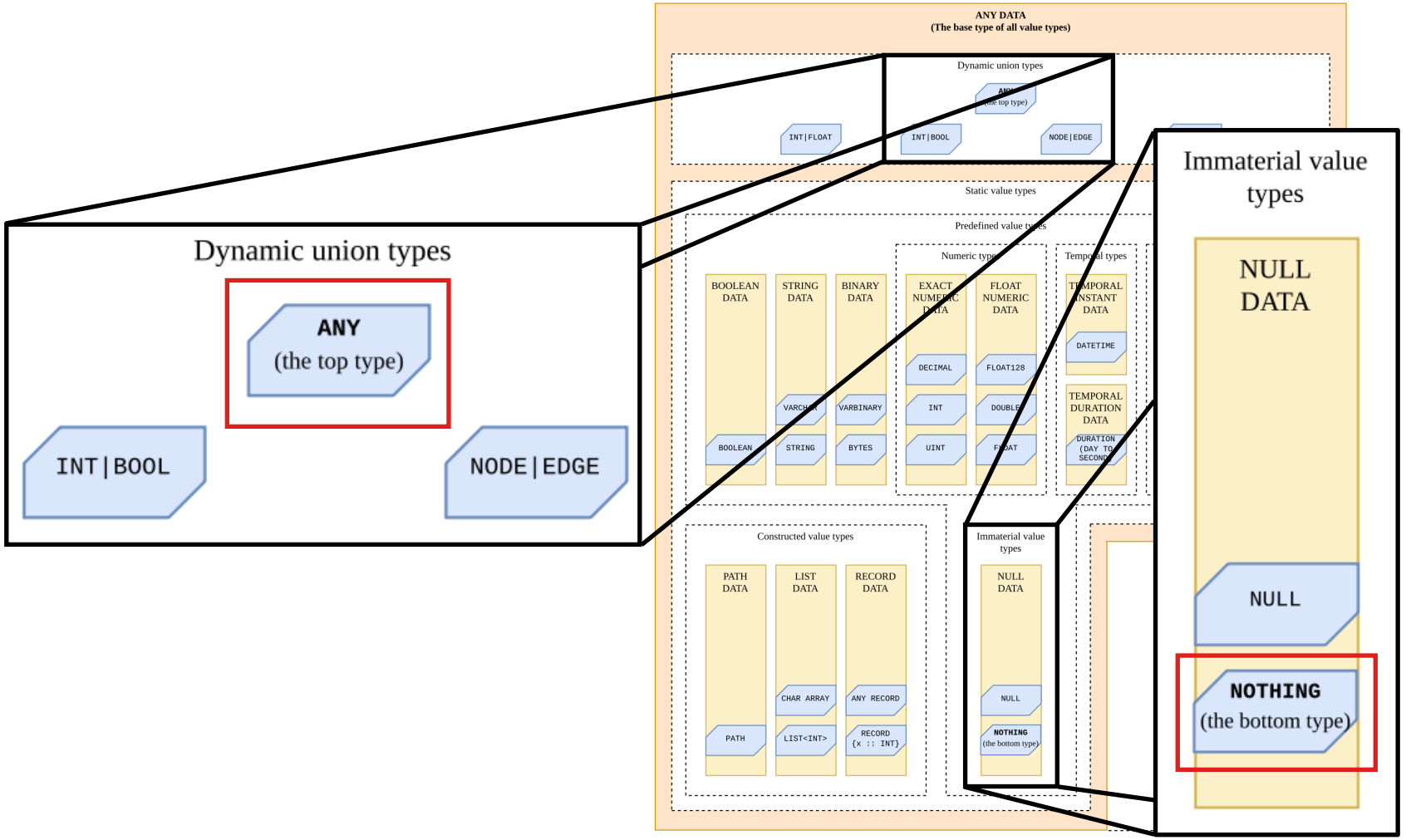
ISO/IEC 39075:2024, Fig. 3 (p. 61)

Type System | The Empty Former



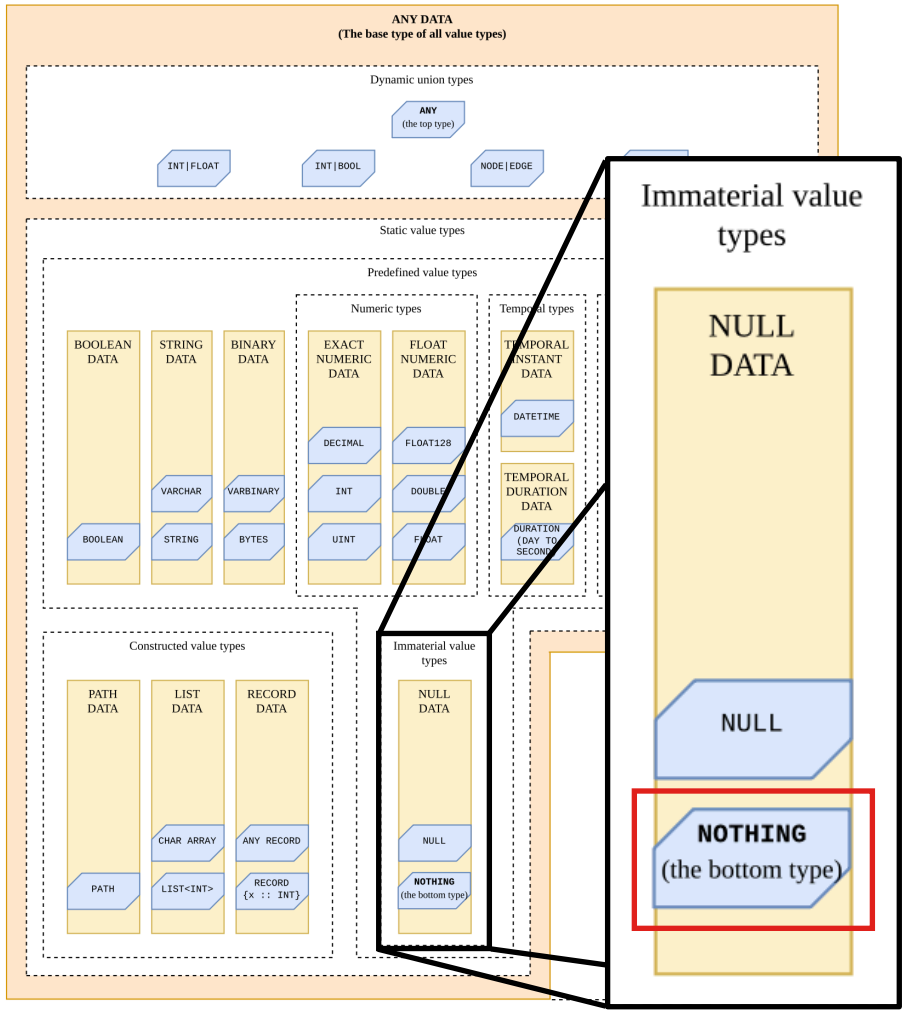
ISO/IEC 39075:2024, Fig. 3 (p. 61)

Type System | The Empty Former



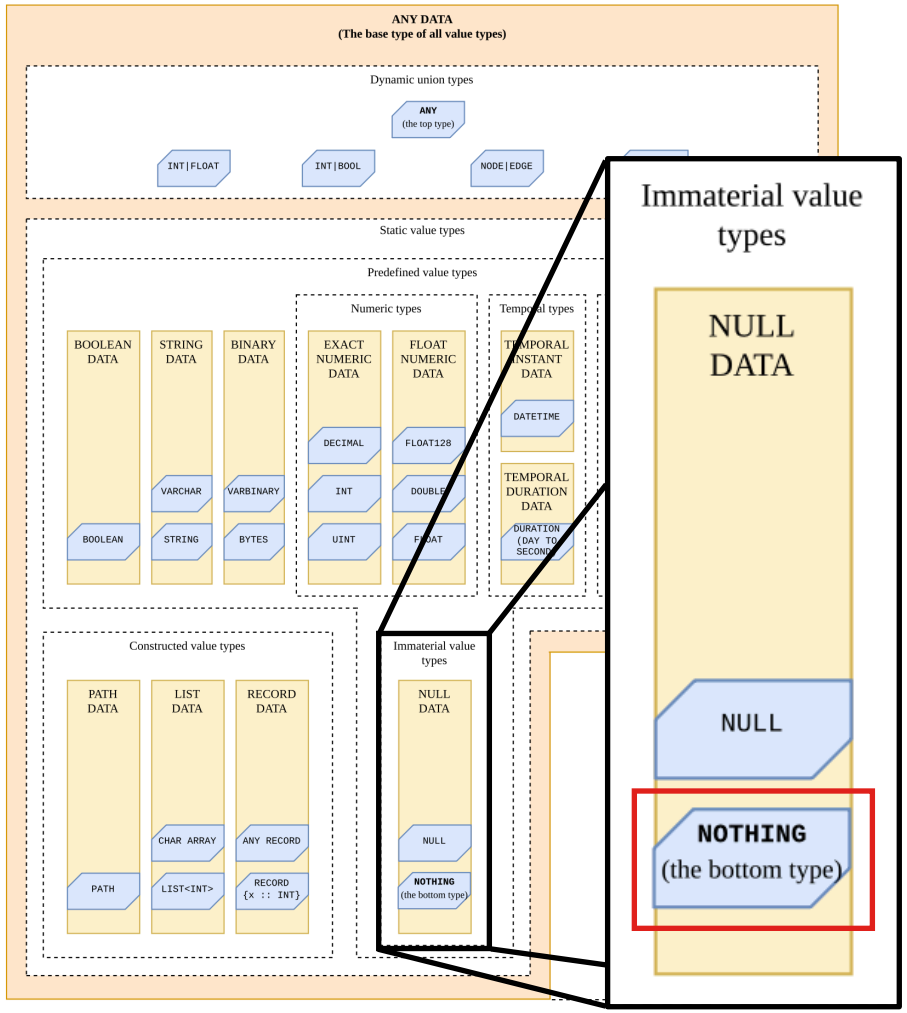
ISO/IEC 39075:2024, Fig. 3 (p. 61)

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

The standard specifies a single empty type: **NOTHING (given by ⊥)**

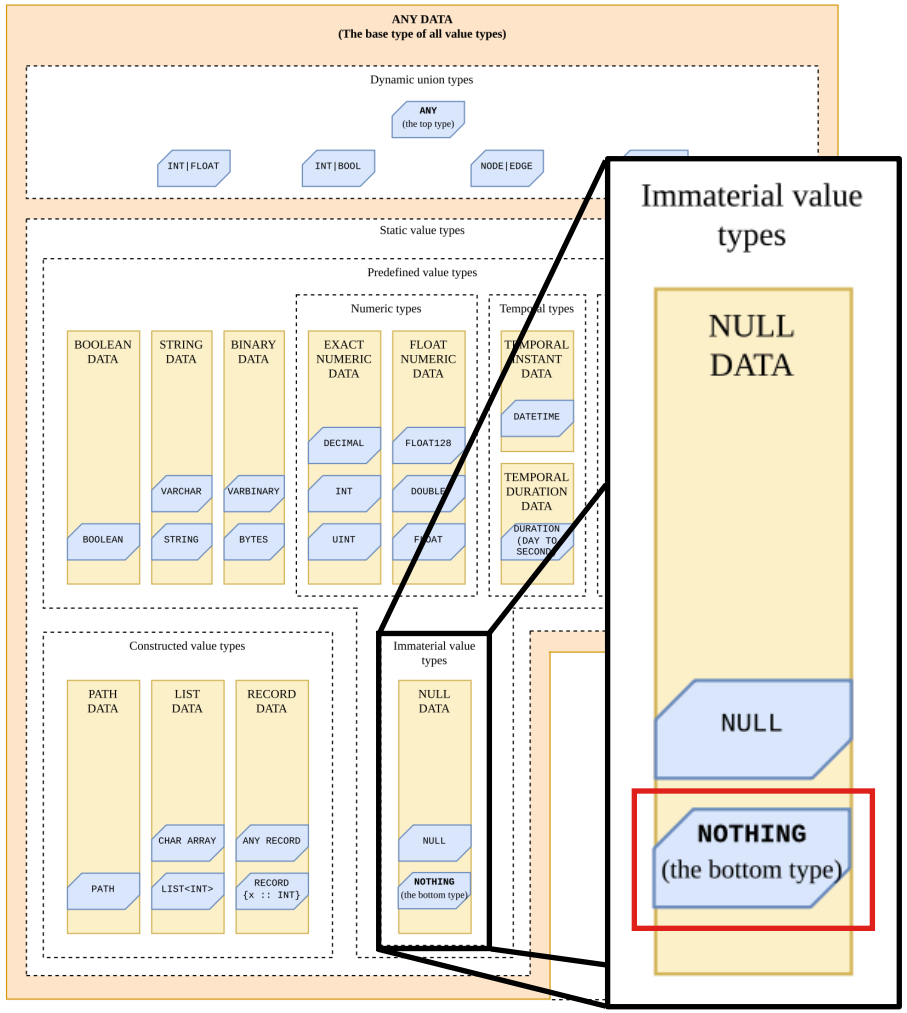
> *"… the empty type comprises no values (is the empty set) …"*
>
> *"… the empty type is a **subtype of every value type**."*
>
> §4.16.8 (p. 94)

Q1 `MATCH (x:Company), (x:Company)-[:WORKS_AT]->()`

WORKS_AT can only start from Person, so x can **never match**

	GQL's empty type	
Q1		

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

The standard specifies a single empty type: **NOTHING (given by ⊥)**

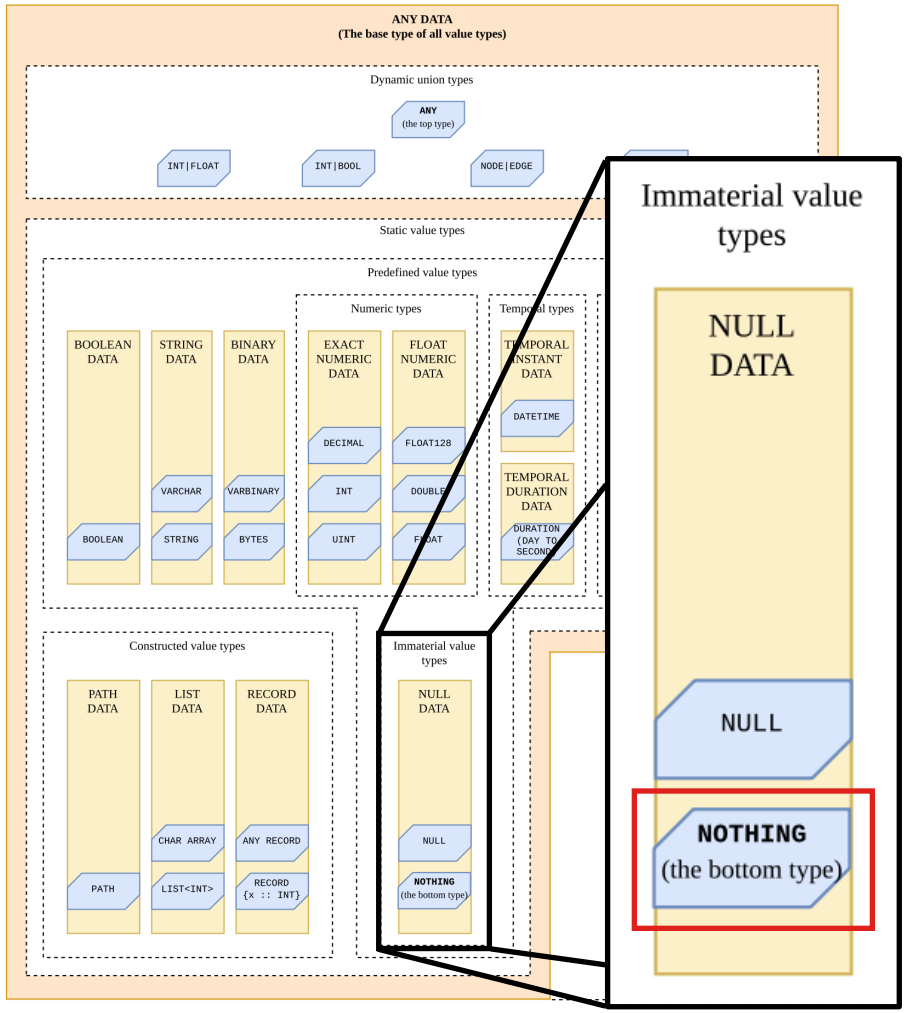
> *"… the empty type comprises no values (is the empty set) …"*
>
> *"… the empty type is a **subtype of every value type**."*
>
> §4.16.8 (p. 94)

Q1 `MATCH (x:Company), (x:Company)-[:WORKS_AT]->()`

WORKS_AT can only start from Person, so x can **never match**

	GQL's empty type	
Q1	`x : ⊥`	

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

The standard specifies a single empty type: **NOTHING (given by ⊥)**

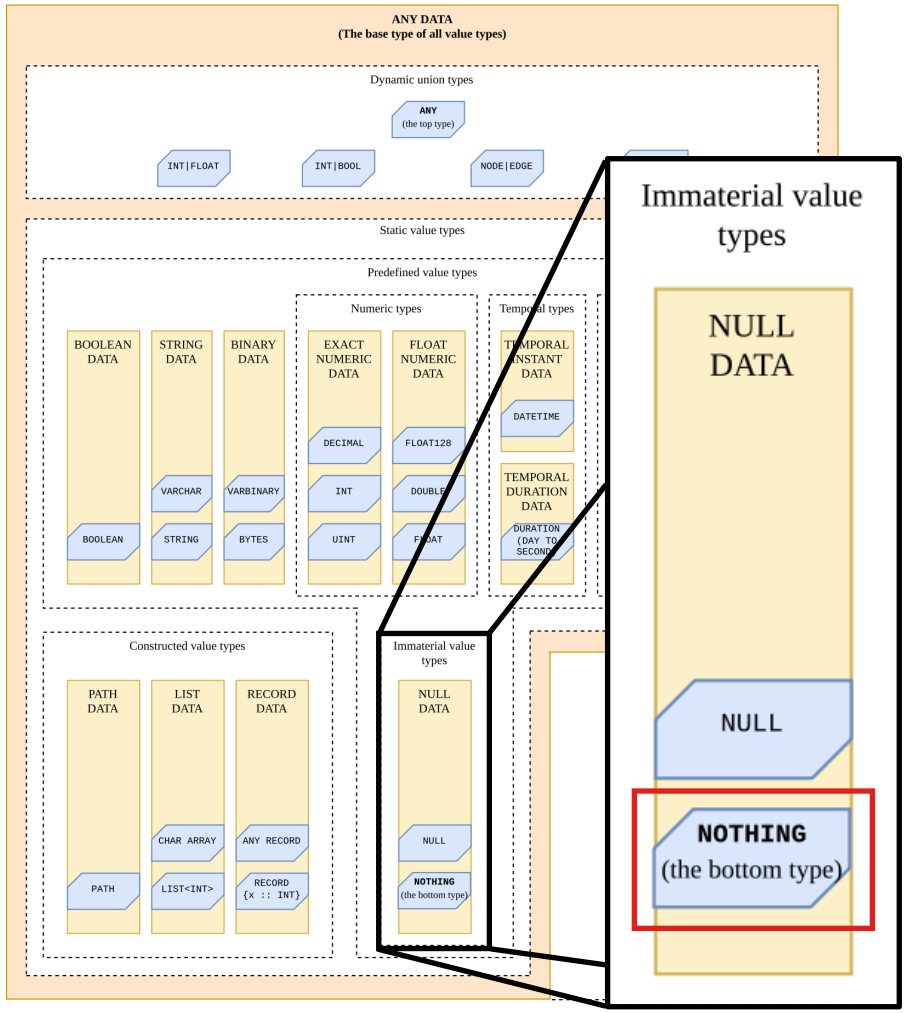
> *"… the empty type comprises no values (is the empty set) …"*
>
> *"… the empty type is a **subtype of every value type**."*
>
> §4.16.8 (p. 94)

Q1 `MATCH (x:Company), (x:Company)-[:WORKS_AT]->()`

WORKS_AT can only start from Person, so x can **never match**

	GQL's empty type	
Q1	x : ⊥ **empty, but a node or an edge?**	

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

The standard specifies a single empty type: **NOTHING (given by ⊥)**

> *"… the empty type comprises no values (is the empty set) …"*
>
> *"… the empty type is a **subtype of every value type**."*
>
> §4.16.8 (p. 94)

Q1 `MATCH (x:Company), (x:Company)-[:WORKS_AT]->()`

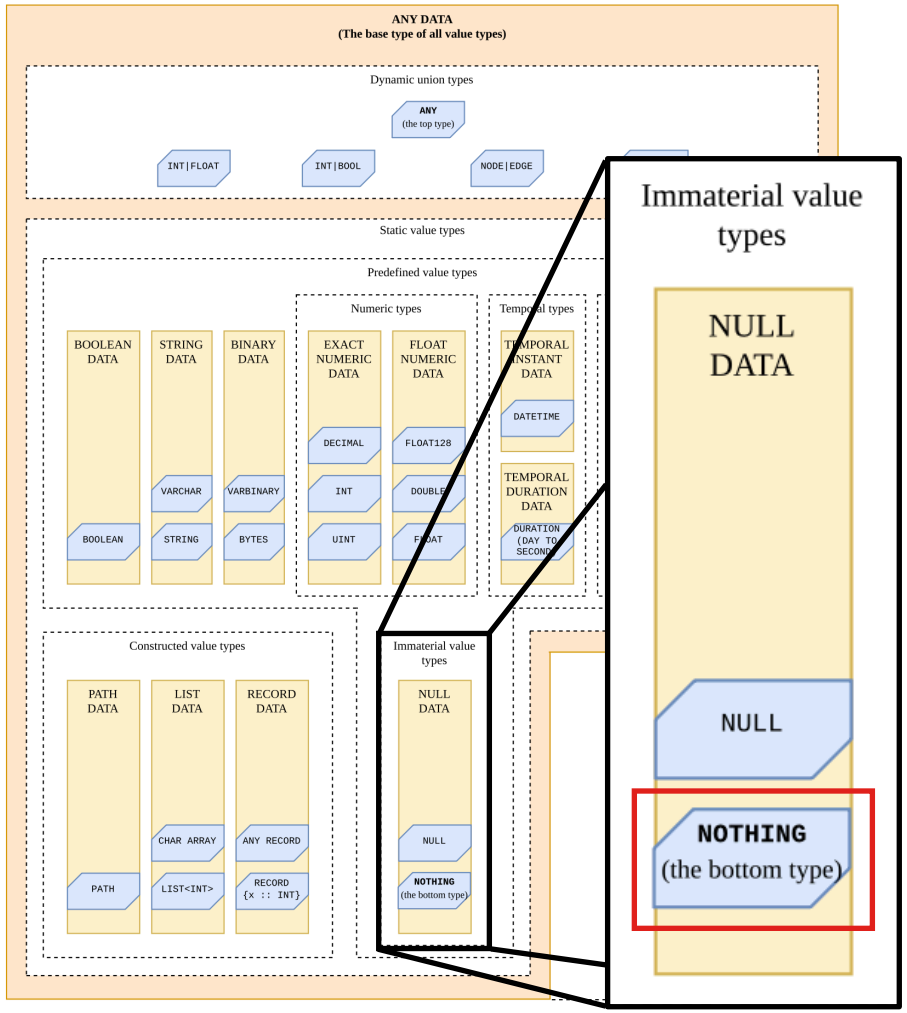
WORKS_AT can only start from Person, so x can **never match**

Q2 `MATCH (x:Company), (:Company)-[x:WORKS_AT]->()`

x is a node *and* an edge, so it **must be rejected** (§16.4 SR 5)

	GQL's empty type	
Q1	$x : \bot$ **empty, but a node or an edge?**	
Q2	$\bot <: N\langle G \rangle$ and $\bot <: E\langle G \rangle$	

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

The standard specifies a single empty type: **NOTHING (given by ⊥)**

> *"… the empty type comprises no values (is the empty set) …"*
>
> *"… the empty type is a **subtype of every value type**."*
>
> §4.16.8 (p. 94)

Q1

```
MATCH (x:Company), (x:Company)-[:WORKS_AT]->()
```

WORKS_AT can only start from Person, so x can **never match**

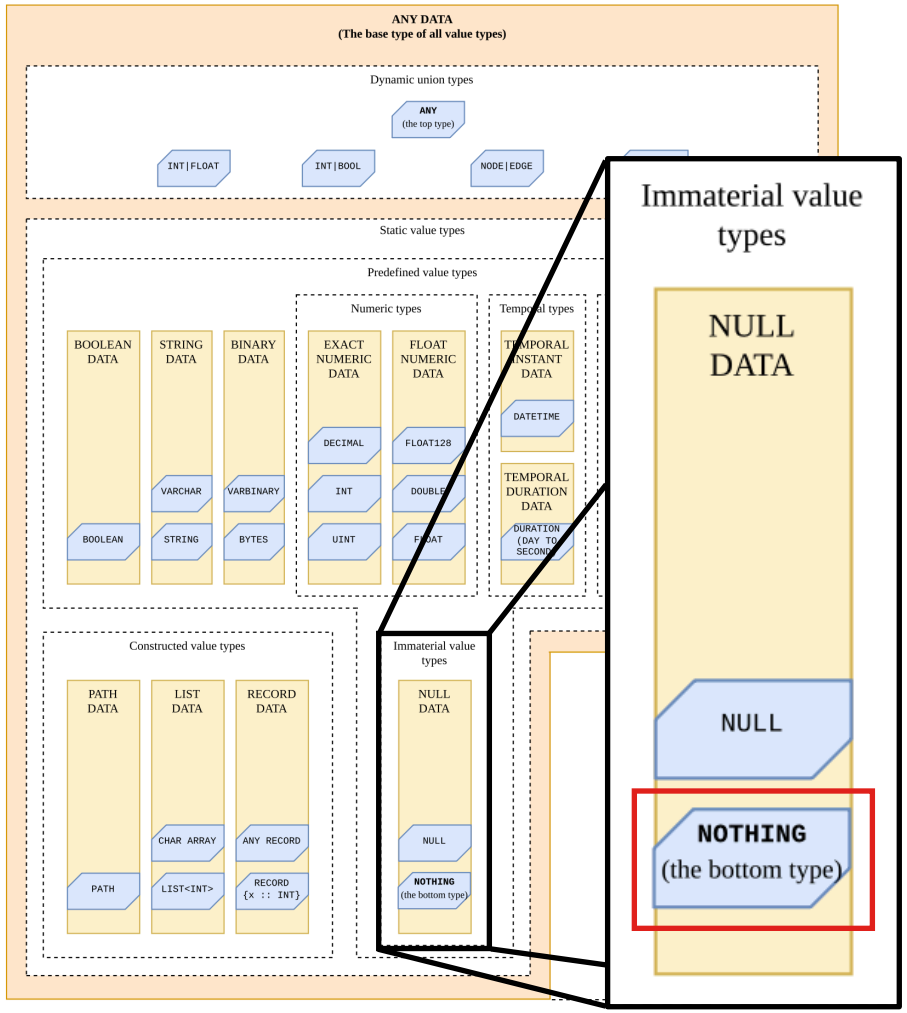
Q2

```
MATCH (x:Company), (:Company)-[x:WORKS_AT]->()
```

x is a node *and* an edge, so it **must be rejected** (§16.4 SR 5)

	GQL's empty type	
Q1	$x : \bot$ **empty, but a node or an edge?**	
Q2	$\bot <: N\langle G \rangle$ and $\bot <: E\langle G \rangle$ **fits both so can't reject**	

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

The standard specifies a single empty type: **NOTHING (given by ⊥)**

> *"… the empty type comprises no values (is the empty set) …"*
>
> *"… the empty type is a **subtype of every value type**."*
>
> §4.16.8 (p. 94)

Q1 `MATCH (x:Company), (x:Company)-[:WORKS_AT]->()`

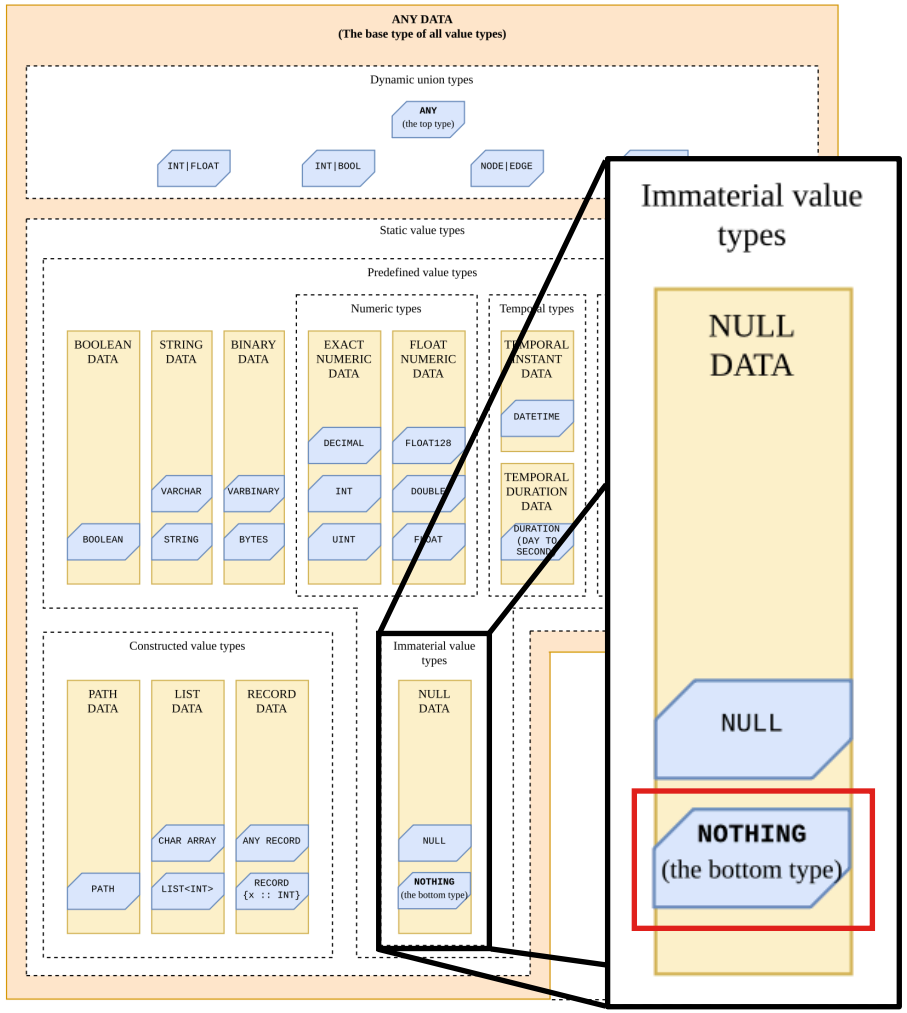
WORKS_AT can only start from Person, so x can **never match**

Q2 `MATCH (x:Company), (:Company)-[x:WORKS_AT]->()`

x is a node *and* an edge, so it **must be rejected** (§16.4 SR 5)

	GQL's empty type	MGQL's empty former
Q1	$x : \bot$ **empty, but a node or an edge?**	$x : N\langle G\rangle^{\bot}$ **empty, and still a node**
Q2	$\bot <: N\langle G\rangle$ and $\bot <: E\langle G\rangle$ **fits both so can't reject**	$N\langle G\rangle^{\bot} <: N\langle G\rangle$ *only* **can be rejected**

Type System | The Empty Former



ISO/IEC 39075:2024, Fig. 3 (p. 61)

The standard specifies a single empty type: **NOTHING (given by ⊥)**

> *"… the empty type comprises no values (is the empty set) …"*
>
> *"… the empty type is a **subtype of every value type**."*
>
> §4.16.8 (p. 94)

Q1 `MATCH (x:Company), (x:Company)-[:WORKS_AT]->()`

WORKS_AT can only start from Person, so x can **never match**

Q2 `MATCH (x:Company), (:Company)-[x:WORKS_AT]->()`

x is a node *and* an edge, so it **must be rejected** (§16.4 SR 5)

	GQL's empty type	MGQL's empty former
Q1	$x : \bot$ **empty, but a node or an edge?**	$x : N\langle G\rangle^{\bot}$ **empty, and still a node**
Q2	$\bot <: N\langle G\rangle$ and $\bot <: E\langle G\rangle$ **fits both so can't reject**	$N\langle G\rangle^{\bot} <: N\langle G\rangle$ *only* **can be rejected**

Empty former captures emptiness while retaining the underlying type

Semantics | Trail & Path Tables

```
USE G MATCH (a {name: 'Megan'})-[x:WORKS_AT]->(c)<-[y:WORKS_AT]-(b) RETURN b.name
```

Semantics | Trail & Path Tables

```
USE G MATCH (a {name: 'Megan'})-[x:WORKS_AT]->(c)<-[y:WORKS_AT]-(b) RETURN b.name
```

The standard only specifies Binding Table

a	x	c	y	b	
M	w1	Versalife	w2	B	kept
M	w1	Versalife	w1	M	x = y: drop

> Then DIFFERENT EDGES, §16.4 General Rule 4 (p. 216):
> *"… there is no edge binding EB2 … at a different position than EB1 that binds the edge E."*

Semantics | Trail & Path Tables

```
USE G MATCH (a {name: 'Megan'})-[x:WORKS_AT]->(c)<-[y:WORKS_AT]-(b) RETURN b.name
```

The standard only specifies Binding Table

a	x	c	y	b	
M	w1	Versalife	w2	B	kept
M	w1	Versalife	w1	M	x = y: drop

> Then DIFFERENT EDGES, §16.4 General Rule 4 (p. 216):
> *"… there is no edge binding EB2 … at a different position than EB1 that binds the edge E."*

Records joined on **c** are unaware about edges, so the path with repeated edges also survives

Semantics | Trail & Path Tables

```
USE G MATCH (a {name: 'Megan'})-[x:WORKS_AT]->(c)<-[y:WORKS_AT]-(b) RETURN b.name
```

The standard only specifies Binding Table

a	x	c	y	b	
M	w1	Versalife	w2	B	kept
M	w1	Versalife	w1	M	x = y: drop

> Then DIFFERENT EDGES, §16.4 General Rule 4 (p. 216):
> *"… there is no edge binding EB2 … at a different position than EB1 that binds the edge E."*

Records joined on **c** are unaware about edges, so the path with repeated edges also survives

MGQL checks while composing with Trail & Path tables

Trail table (R, W) **Path table** (R, n_l, n_r, W)

$$(R_1, n_l, m, W_1) \cdot (R_2, m, n_r, W_2) = (R_1 \bowtie R_2, n_l, n_r, W_1 \cup W_2)$$

built only if the ends meet (the shared m) **and** $W_1 \cap W_2 = \varnothing$

(a)-[x]->(c)

n_l	n_r	W
M	Versalife	{w1}

$\cdot$

(c)<-[y]-(b)

n_l	n_r	W
Versalife	M	{w1}
Versalife	B	{w2}
UNATCO	A	{w3}

=

candidates

- Megan → Versalife → Bob **built**
- Megan → Versalife → Megan **shares w1, so not built**
- Versalife ≠ UNATCO **never built**

Mechanization | A Lean4 Library

14 modules **0** sorry, **0** user axioms **350** executable assertions **6** LDBC SNB queries

```
-- ① a query is a Lean term
def q : Query :=
  .useGraph "g" (.matchWhere
    (.edge {var := "a", labels := some (.atom "Person")}
           {var := "k", labels := some (.atom “WORKS_AT”)} .right
           {var := "b", labels := some (.atom "Person")})
    (.relOp .gt (.propAccess "a" "age") (.const (.int 30)))
    [.alias' (.propAccess "b" "name") "friend"])

-- ② type checker: the result schema
#eval inferQuery ctx q

-- ③ executable semantics: the result table
#eval evalQuery ctx.catalog G "g" q

-- ④ both engines give the same table
example :
      runQuery .smallStep ctx.catalog G "g" q
  = runQuery .bigStep   ctx.catalog G "g" q := by native_decide
```

Easily Extensible

① Encode graphs, schemas and queries as plain Lean terms

② **inferQuery**: a type checker proven sound against the typing rules

③ **evalQuery**: run any query in the fragment

④ **runQuery_engine_agnostic**: big-step and small-step engines always agree

Contributions to the GQL Standard

Contributions to the GQL Standard

Inconsistency | Nested Quantifiers

MATCH ((()-[e]->())* -[]->(f))*

§16.7 Syntax Rule 9 (p. 229) says: illegal

"The <path primary> simply contained in a <quantified path primary> shall not contain a <quantified path primary> at the same depth of graph pattern matching."

…and nesting stays "at the same depth" (§16.4 SR 2, NOTE 216)

But

§16.4 NOTE 221 (p. 215) uses it as an example

(((-[E]->) -[E]->)* -[F]->)*

Contributions to the GQL Standard

Inconsistency | Nested Quantifiers

MATCH ((()-[e]->())* -[]->(f))*

§16.7 Syntax Rule 9 (p. 229) says: illegal

"The <path primary> simply contained in a <quantified path primary> shall not contain a <quantified path primary> at the same depth of graph pattern matching."

…and nesting stays "at the same depth" (§16.4 SR 2, NOTE 216)

But

§16.4 NOTE 221 (p. 215) uses it as an example

(((-[E]->) -[E]->)* -[F]->)*

Proposal | Empty Former

GQL Standard: emptiness captured only by Empty type

$\perp$

Proposed: emptiness that retains the underlying type

$N\langle G\rangle\ \perp,\quad E\langle G\rangle\ \perp$

Statically propagates query emptiness in more situations, without compromising well-formedness checks

Useful for query optimizers and fuzz / metamorphic testing of graph databases [GraphGenie, Jiang et al. ICSE 2024]

Contributions to the GQL Standard

Inconsistency | Nested Quantifiers

MATCH ((()-[e]->())* -[]->(f))*

§16.7 Syntax Rule 9 (p. 229) says: illegal

"The <path primary> simply contained in a <quantified path primary> shall not contain a <quantified path primary> at the same depth of graph pattern matching."

…and nesting stays "at the same depth" (§16.4 SR 2, NOTE 216)

But

§16.4 NOTE 221 (p. 215) uses it as an example

(((-[E]->) -[E]->)* -[F]->)*

Proposal | Empty Former

GQL Standard: emptiness captured only by Empty type

$\perp$

Proposed: emptiness that retains the underlying type

$N\langle G\rangle\ \perp,\quad E\langle G\rangle\ \perp$

Statically propagates query emptiness in more situations, without compromising well-formedness checks

Useful for query optimizers and fuzz / metamorphic testing of graph databases [GraphGenie, Jiang et al. ICSE 2024]

Positive response from the standards community

Conclusions

The first foundation for rigorous formal reasoning of ISO GQL's semantics

1 **Schema-aware type system**
with an Empty Former that captures emptiness and retains the base type

2 **Small-step semantics**
for the read-only pipeline: bags, nulls, quantified paths, composite queries

3 **Mechanization in Lean 4**
which exposed an ambiguity in the standard



MGQL received an
ACM SIGPLAN Distinguished Artifact Award

MGQL: github.com/EngineeringSoftware/mgql

Contact: Aditya Thimmaiah auditt@utexas.edu